吉林大学

软件学院

《面向对象程序课程设计》
实验报告

实验地点	计算机大楼实验室	**座位号**	
班级	**2 班**		
学号	**55250229**		
姓名	**李佳烨**		

2026—2027 学年第 1 学期

实验项目	**模拟即时通信系统实现**

一、设计任务分析

1 系统目标与对象识别

本项目使用 C++ 实现模拟即时通信系统，管理 QQ、微信和微博三项模拟服务中的账号、好友与群关系，不连接真实社交平台。设计以人员、服务账号、好友关系、群和群成员为主要对象，体现抽象、封装、继承、组合与运行时多态；数字菜单作为课程业务入口，Qt 界面、SQLite 和本机通信作为扩展。

账号支持注册、登录、资料修改及服务开通。QQ 与微博共享对外号码，微信使用独立号码并可绑定 QQ。好友支持申请、同意、备注、删除、共同好友和跨服务推荐；群支持申请或邀请、成员维护、管理员任免、群主转让、一级子群及管理模式切换。

2 设计重点与难点

重点是区分“人”和“账号”：同一人可开通多项服务，人员资料不应在每项服务中重复保存。好友关系与群内身份属于服务账号，不能只凭一个数字号码判断对象。另一难点是群的成员数据保持不变时，如何切换 QQ 与微信权限；设计应把稳定的数据与变化的规则分开，而不是复制两套群数据。

二、设计方案

1 类、对象与关联关系

PersonLJY 表示人员，保存姓名、生日和所在地；ServiceAccountLJY 表示一项已开通账号，保存服务类型、对外号码、昵称和申请日期，并以 personId 关联人员。两者是关联关系，不是继承关系：账号并不是一种人。同一个类可以创建许多对象，增加用户只增加对象和数据，不增加新的类文件。

FriendRelationLJY 用两个内部账号编号表示一对双向好友，并分别保存双方备注，避免用两条独立记录造成关系不一致。GroupLJY 拥有 GroupMemberLJY 成员集合；成员对象记录账号编号和管理员标记，同一账号在不同群可以具有不同身份。图 1 给出主要关系，编号关联不代表直接持有对方对象。

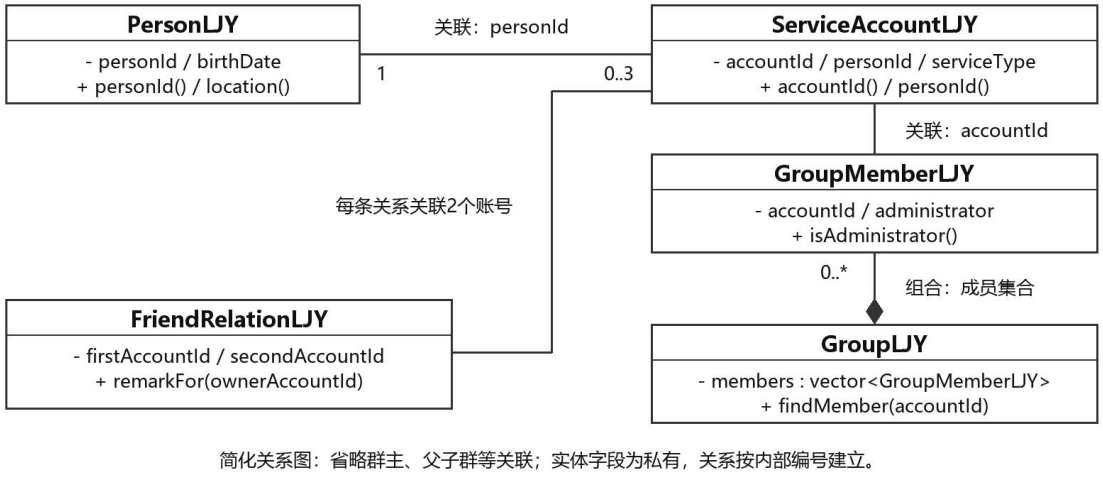


图 1　核心业务对象关系

2 封装、组合与职责划分

实体数据设为 private，并通过只读方法查询；例如 FriendRelationLJY::remarkFor 返回某一方的备注，修改只能经好友业务检查后进行。PlatformLJY 私有地持有人员、账号、关系和群容器，向外提供查询接口，并向必要的业务管理器及存储类授予 friend 访问权。封装不只是隐藏字段，还包括限定修改入口和维护对象间约束。

SessionManagerLJY 按值组合 AccountServiceLJY、FriendManagerLJY 和 GroupManagerLJY，保存当前登录身份并组织业务调用。会话和管理器通过引用使用同一个 PlatformLJY，平台由程序入口创建，不能将这种引用误画成生命周期归属的组合。界面只收集输入和显示结果，终端与图形端复用业务接口。

3 抽象接口、继承与多态

SocialServiceLJY 以纯虚函数声明服务类型、名称、号码模式和默认群规则，QQServiceLJY、WeChatServiceLJY、WeiboServiceLJY 分别实现。PlatformLJY 用 unique_ptr<SocialServiceLJY> 统一持有派生对象，调用虚函数时执行实际对象的实现；基类虚析构函数支持通过基类指针释放派生对象。这里的继承表达“某种具体服务是一种社交服务”。

群规则采用策略模式。GroupManagerLJY 持有两种策略对象，policy(style) 返回 GroupPolicyLJY 基类引用，再调用 canApply、canKick 等虚函数。同一个调用接口因实际策略对象不同而得到不同结果。图 2 中空心三角指向基类，虚线箭头表示依赖。

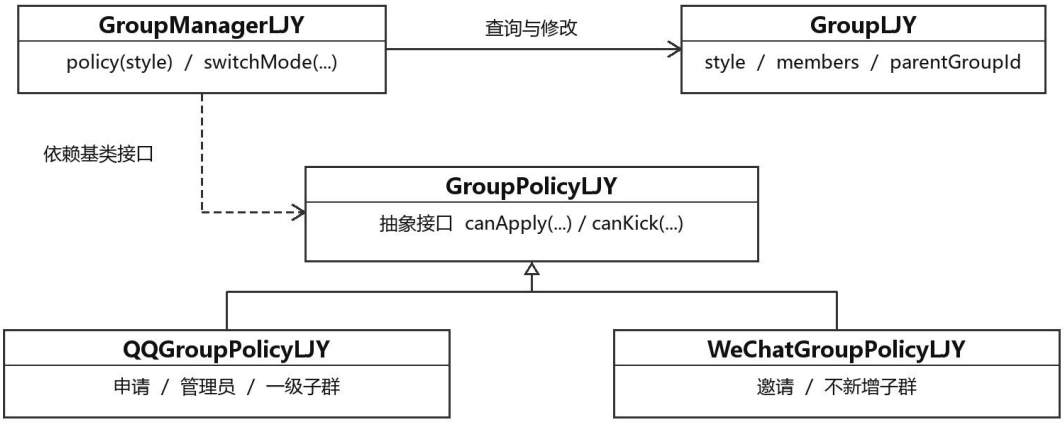


图 2　群策略与运行时多态

GroupLJY 保存数据，QQGroupPolicyLJY 与 WeChatGroupPolicyLJY 判断权限；群本身不继承策略。切换模式只修改 style，既有成员、管理员标记和子群保留，后续调用转向另一策略。微信模式不能新建子群或任免管理员，旧管理员标记不赋予管理特权；已有子群仍按自身模式管理。微博默认采用 QQ 群规则。

三、详细设计

1 对象建立与容器管理

项目采用 C++17、CMake 和 Ninja；图形发布版使用 Qt 6.8.3 与 MinGW 13.1。入口创建 PlatformLJY，读取数据后构造 SessionManagerLJY，再进入数字菜单或 Qt 事件循环。新普通库初始化演示对象，已有库恢复原数据；内存模式不写库。

注册由账号业务检查资料和两次密码输入，再建立 PersonLJY 与首项 ServiceAccountLJY。之后开通服务复用人员编号，按服务规则选择共享号码或独立发号。内部 accountId 唯一定位平台中的账号；对外号码则必须与服务类型一起使用。类名统一使用作者姓名缩写 LJY，与注册用户姓名无关。

PlatformLJY 以 map 保存人员、账号及群，好友关系以排序后的账号编号对为键；GroupLJY 以 vector 保存成员值对象。平台销毁时容器成员随之释放，服务派生对象由 unique_ptr 管理。对象间多数关系以编号表达，避免为同一人员或账号反复复制完整资料；返回的查询指针仍由平台管理。

2 对象协作与业务约束

以添加好友为例：界面传入目标号码，SessionManagerLJY 取得当前登录账号，FriendManagerLJY 查找同服务目标并建立申请；对方同意后才生成 FriendRelationLJY。修改备注只改变操作者对应的字段。这条调用路径把输入显示、当前身份、关系规则与数据保存分开。

以移除群成员为例：会话确定执行者，GroupManagerLJY 查找群和目标成员，按当前策略检查权限，再修改成员集合及相关请求。QQ 模式下群主移除管理员前须先取消

其管理员身份；微信模式只有群主能移除其他成员。父群成员若仍是某个子群群主，则整项移除被拒绝，防止产生没有合法归属的子群身份。

动态切换的关键不是把一个 C++ 对象强行转换为另一种类型，而是让同一个群对象改用另一组权限规则。QQ 模式可申请入群，切到微信后改为邀请加入；数据与行为选择分离，因此不必重建群和成员。

3 界面、存储与通信的扩展边界

Qt 页面按账号、好友和群等业务组织，操作结果由业务层返回，权限不依赖按钮是否可点击。SqliteStorageLJY 封装业务快照的读取与事务保存，图形端提交后自动保存，终端保留保存菜单及正常退出保存。业务对象不持有控件，也不负责拼接 SQL，使显示方式和保存方式可以独立调整。

通信扩展中，ChatProtocolLJY 封装长度分帧，TcpPeerLJY 封装 Winsock 和后台线程；群服务封装多人转发与持久日志。业务快照、客户端消息历史、服务端日志分开保存。当前仅验收 Windows 本机通信；群离线补齐依赖服务已存日志，私聊不支持离线投递。

4 调试与验证

调试关注对象状态是否一致：双向好友只建立一次、双方备注互不覆盖、群模式切换不丢成员、子群成员仍属于父群。群历史缺失则通过独立日志与同步解决；浏览好友与实际连接对象分别记录，避免切换页面导致连接状态混乱。既有验收覆盖这些业务与扩展流程；2026 年 10 月 21 日的 18 项相关测试以及 10 月 25 日的子群选择、保存重载和缩放检查均有日志，本轮不重复运行。

四、总结与体会

面向对象设计首先需要明确对象含义与责任：人员不是账号，好友关系不是人员的子类，群内身份也不能写成账号的唯一属性。关联表达对象间联系，组合表达组成和生命周期，继承表达共同接口下的具体类型；恰当选择这些关系，比增加类的数量更重要。

封装让资料和关系通过受控入口修改；抽象接口统一服务能力；运行时多态让群规则能够切换；会话组合管理器使界面与业务分工明确。调试也说明，一次操作显示成功并不代表关联数据已一致，必须同时考虑对象约束和保存结果。

现有实现仍有不足：部分图形页面较大，扩展模块有辅助类型未独立同名文件。这 14 个扩展辅助类型中，界面 5 个、数据库 3 个、通信及聊天身份 6 个；人员、账号、好友、群及核心管理器已独立。文件规范差异仍应如实保留，不能据核心业务已独立就声称全部满足；后续优先做轻量整理，不增加无必要抽象。